

Docker

Containers, Dockerfiles and Docker Compose

Birgit Schröder

2026-09-17

Table of contents

Introduction	3
1 Containers	6
1.1 Container commands	7
2 Dockerfiles	10
2.1 Docker Images	11
2.2 First Dockerfile	14
2.3 Publish Static Web Page	15
2.4 Javascript Express Backend	16
2.5 Multistage Dockerfiles	17
2.6 TypeScript Backend Dockerfile	19
2.7 Hosting Dynamic Website with nginx	21
3 Docker Compose	23
3.1 Container Orchestration with <code>docker compose</code>	24
3.2 Composing a single container	25
3.3 Exercise: Compose container network	29
3.4 DB Containers / Persistent Volumes	31
3.5 Wordpress with docker compose	34

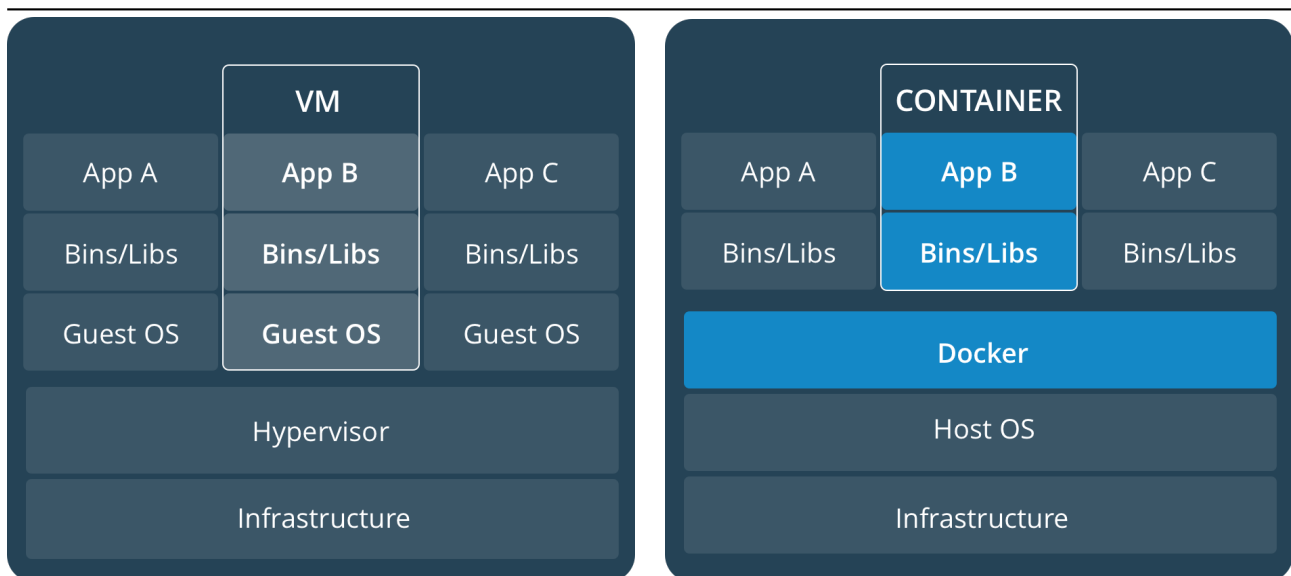
Introduction

Introduction to [Docker](#)

What is *Docker*?

- Product/Company that made *Container* technology very popular
- Products
 - Docker Engine: the open-source container runtime and CLI (free)
 - Docker Desktop: GUI application including Docker Engine for Windows, macOS and Linux (free for personal use, education and small businesses; larger companies need a paid subscription)
- Is available on all major platforms
- Learn more in [Docker docs...](#)

Docker versus Virtual Machine



In classical **Virtual Machines** (VM),

- each VM runs its own guest operating system
- consuming **tens of GB** of data

Containers

- **reuse** the host operating system
- run in user space
- consume only **tens of MB** of data

[Image Source and more information...](#)

Installation

Docker Desktop (Windows)

- [WSL2 \(Windows Subsystem for Linux - Version 2\)](#)
- [Docker Desktop for Windows](#)

Docker on Linux Ubuntu

- [Install docker with convenience script](#)
 - Fine for learning and test environments, but not recommended by Docker for production systems (use the `apt` repository there)

- [Post installation step](#): Add your user (eg. `ubuntu`) to the `docker` group, so you can run docker without `sudo`
 - `sudo usermod -aG docker ubuntu`
 - `newgrp docker`
- Test with `docker run hello-world`

Which terminal should I use? (Windows)

Use **PowerShell** (e.g. the VS Code terminal) for all exercises. Docker Desktop must be running.

If you use **Git Bash** instead, two things go wrong:

- `docker run -it ...` and `docker exec -it ...` fail with the input device is not a TTY. Fix: put `winpty` in front of the command, e.g. `winpty docker run -it ubuntu bash`.
- Paths like `/bin/bash` are converted into Windows paths. Fix: write `bash` instead of `/bin/bash`, or write `//bin/bash`.

Docker Terms

Term	Description	Main commands
Docker Image	A read-only template which contains everything needed to run an application as a container. Consists of several layers.	<code>docker images</code> lists all images available in the local image cache (pulled or built before).
Docker Container	A single runtime instance of a docker image. From one image, a lot of containers can be started and stopped, with individual execution parameters (e.g. for the port mapping of a container).	<code>docker ps</code> lists all running docker containers <code>docker ps -a</code> including stopped containers
Docker Registry	A registry stores and distributes docker images. Docker Hub is the default registry and hosts a huge number of public images including usage documentation and version histories. It can also be used for managing private docker images. Besides Docker Hub, there are a lot of other container registries, e.g. the GitHub Container Registry (ghcr.io).	<code>docker pull</code> , <code>docker push</code>
Dockerfile	A text file containing commands for building a new docker image. Instructions that change the file system (like <code>RUN</code> and <code>COPY</code>) each add a new image layer. Dockerfile syntax uses instructions like <code>FROM</code> for referring a base image, <code>RUN</code> for executing a shell command, <code>COPY</code> for copying files from the local file system into the image, and <code>CMD</code> for specifying the start command of the application which will be executed whenever a container from that image is started.	<code>docker build -t <image-name> .</code>
Docker Compose	A tool for defining and running multi-container applications in a YAML file	<code>docker compose up</code> <code>docker compose down</code>

Chapters

- [Containers](#)
- [Building images from Dockerfiles](#)
- [Container Orchestration with docker compose](#)

Cleaning up

Clean up after each exercise. Otherwise, old containers keep using ports and names, and the next exercise fails.

What	Command
Show all containers (also stopped ones)	<code>docker ps -a</code>
Stop and remove a container	<code>docker rm -f <name-or-id></code>
Stop and remove a compose project	<code>docker compose down</code> (in the compose folder)
Remove all stopped containers, unused networks and dangling images	<code>docker system prune</code>

Troubleshooting

Error message	Cause	Fix
Cannot connect to the Docker daemon / error during connect the input device is not a TTY	Docker Desktop is not running You use Git Bash	Start Docker Desktop and wait until it says “Engine running” Use PowerShell, or put <code>winpty</code> in front of the command (see above)
port is already allocated / address already in use	Another container (or program) already uses this host port	<code>docker ps</code> → remove the old container, or choose another host port (left side of <code>-p</code>)
The container name “/...” is already in use	A container with this name still exists (maybe stopped)	<code>docker rm -f <name></code> , or <code>docker compose down</code> in the folder of the other exercise
Port 3306 or 80 is taken although no container runs Browser shows nothing / “connection refused”	A local program (e.g. XAMPP MySQL/Apache) uses the port Wrong port, or the container has stopped	Stop that program, or use another host port Check the PORTS column in <code>docker ps</code> ; check <code>docker logs <name></code>

Part 1

Containers

1.1 Container commands

Stuck or have a question? Ask the [AI tutor for this chapter](#). It knows this chapter and helps you with commands and error messages. You can ask in German or English.

Operation	Docker CLI command
Start a new container	<code>docker run</code>
List running containers	<code>docker ps</code>
Stop container	<code>docker stop</code>
Start a stopped container	<code>docker start</code>
Attach to a running container	<code>docker attach</code>
Displays processes in container	<code>docker top</code>
Run process in container	<code>docker exec</code>
Remove container	<code>docker rm</code>

1.1.1 Interactive container

```
docker run --name helloDocker -i -t ubuntu /bin/bash
|
| | | |
| | | | +-- Command to run
| | | +----- Image to start the container from
| | +----- Allocate a pseudo-TTY
| +----- Keep STDIN open (-i -t is usually written as -it)
+----- Name of container
```

1.1.1.1 Exercise:

- Run this command => ubuntu image will be downloaded to the local image cache and a bash prompt is opened.
- Inspect the running ubuntu instance via the bash terminal, for example:
 - `hostname` - which name does the container have?
 - `cat /etc/os-release` - which Linux version is running inside?
 - `ps aux` - which processes are running? (only very few - compare with your own computer!)
 - `ls /` - the container has its own file system
- Use another terminal to inspect
 - the running docker instance with `docker ps` - note the **container id** is the same as displayed in the bash prompt => by default, the (short) container id is used as the host name of the container.
 - `docker images` will display the downloaded ubuntu image.
- Exit the container instance with typing `exit` in the bash prompt.
 - This also **stops** the container: a container only runs as long as its main process (here: `bash`) runs.
 - Use `docker ps -a` to list all containers, including stopped ones.
 - Use `docker start <container-id>` to restart the stopped container. You can see it now running again with `docker ps`, however you are not connected with the bash terminal.
 - Use `docker attach <container-id>` to attach your terminal to the bash prompt again.

1.1.2 Background process

```
docker run -d ubuntu /bin/bash -c "while true; do echo hello world; sleep 1; done"
|
+----- Detached (= background server process)
```

1.1.2.1 Exercise:

- Inspect the running container with `docker ps` => note the container-id.

- Use `docker logs <container-id>` to view the output of the bash command that is running in the container.
- Use `docker stop <container-id>` to stop this container.
- Use `docker ps -a` to list the stopped container as well as all other running containers.
- Use `docker rm <container-id>` to remove the stopped container.

1.1.2.2 Notes

- It is sufficient to enter only the first few characters of the container id - as long as this prefix is unique and docker is thus able to identify the container.
- You can remove a still running container by using the **force** option: `docker rm -f <container-id>`.

1.1.3 Nginx Webserver

`docker container run` is the long form of `docker run` - both commands do the same.

```
docker container run -d -p 8080:80 --name my-webserver nginx
```

			+----- Image, from which the container is started
		+-----	Option to specify the container name
	+-----		Publish container port 80 (where nginx listens)
			on port 8080 of the host
+-----			detached mode (running in background)

1.1.3.1 Exercise

- Run this command
- List the container with `docker ps`: It should display
 - the correct port mapping from the host port 8080 to the container port 80
 - as well as the image `nginx`
 - and the requested container name `my-webserver`
- Check your image cache with `docker images` (nginx image should have been pulled from Docker Hub)
- Connect to the web service from your favourite browser via `http://localhost:8080`. It should display

```
C:\Users\birgi>docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS
d6b868a729d5   nginx    "/docker-entrypoint..."  4 minutes ago  Up 4 mi
```

localhost:8080

Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.

the default `index.html` for nginx:

- Check the logging of your client connection with `docker logs my-webserver` (we can now use the container name instead of the container-id)
- Check the processes running inside this container with `docker top my-webserver`
- Stop and restart the web service with `docker stop` and `docker start`
- Stop the web service again with `docker stop my-webserver`, then remove it with `docker rm my-webserver` (`docker rm` refuses to remove a running container)

1.1.4 Bind Mounts

For publishing a local html folder in your nginx container, you need to specify a **bind mount** with the `-v` option (alternatively: `--mount type=bind,source=...,target=...`):

local folder

volume mapping

```
docker container run -d -p 8080:80 -v C:/htl/docker/wwwroot:/usr/share/nginx/html nginx
```

container folder

Note the colon `:` between the local folder and the folder inside the container!

If your local folder looks like following, your web browser will display your individual `index.html` file after connecting to this new web service:

```
C
+- htl
  +- docker
    +- wwwroot
      +- index.html
```

The command from the picture, to copy (with port 8081, so it does not collide with `my-webserver` from above):

```
docker run -d -p 8081:80 -v C:/htl/docker/wwwroot:/usr/share/nginx/html --name my-bind-webserver nginx
```

Tip: This repository already contains a folder `01-containers/wwwroot` with an `index.html`. If you open a **PowerShell** terminal in the `01-containers` folder, you can use it with `${PWD}` (= current folder):

```
docker run -d -p 8081:80 -v ${PWD}/wwwroot:/usr/share/nginx/html --name my-bind-webserver nginx
```

(On Linux / macOS, write `$(pwd)` instead of `${PWD}`.)

1.1.4.1 Exercise:

- Create folder structure + `index.html` file (or use the `wwwroot` folder of this repository)
- Start a new container with a bind mount and a new port mapping - e.g. with 8081 instead of 8080
- Contact this web service with your favourite browser: `http://localhost:8081` should show **your** `index.html`, not the nginx default page
- Use `docker inspect` to inspect the properties of the new container
- Use `docker stats` to inspect the performance statistics
- Use `docker logs` to access the web server log messages
- Use `docker exec -it <id> bash` to connect to the container in interactive tty mode with a bash terminal. Navigate to the `/usr/share/nginx/html` folder to inspect the mounted files.
- Edit the `index.html` file on your host file system, save it and refresh your browser to see the changed content.
- Add additional files to the mounted folder and check, if they are reachable via your web browser.
- Clean up: `docker rm -f my-bind-webserver`

1.1.5 Check yourself

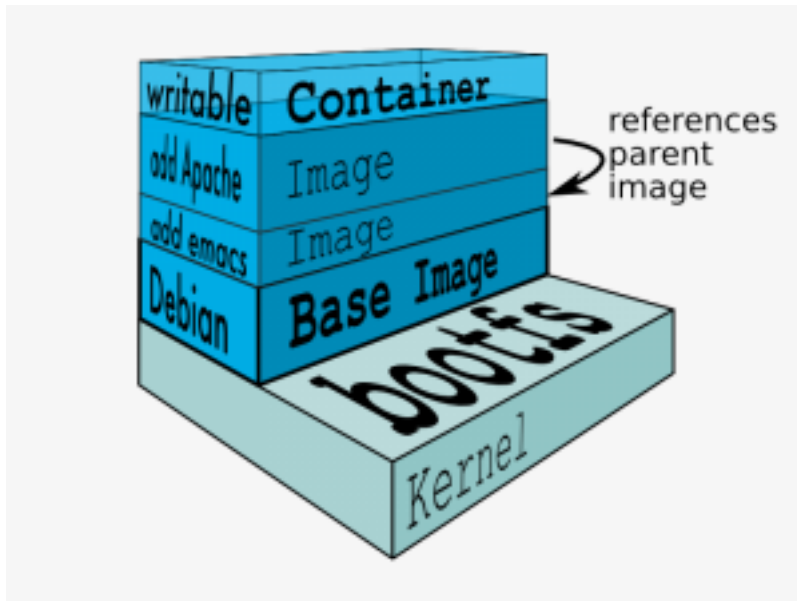
When you have finished all exercises, take the [self-check quiz for this chapter](#): 5 short questions, answered in your own words (German or English). The AI gives you feedback on each answer, and you can ask follow-up questions. You can repeat the quiz as often as you like.

Part 2

Dockerfiles

2.1 Docker Images

Stuck or have a question? Ask the [AI tutor for this chapter](#). It knows this chapter and all its exercises, and helps you with Dockerfiles, builds and error messages. You can ask in German or English.



2.1.1 Image Operations

Operation	Docker CLI command
List images	<code>docker images</code>
Search images	<code>docker search</code>
Pull image from Docker Hub	<code>docker pull</code>
Push image to Docker Hub	<code>docker push</code>

2.1.2 Important Images

Images important for this course

Image	Description
<code>ubuntu</code>	Ubuntu Linux OS
<code>node</code>	Base image for Node.js applications
<code>nginx</code>	Webserver, reverse proxy, load balancer, etc.
<code>eclipse-temurin</code>	Base image for Java applications (JDK and JRE)
<code>mysql</code>	MySQL database

2.1.3 Create Image from a Container

Unusual, but possible: Modify a container and save it to a new image.

2.1.3.1 Step 1: Nginx with Live-Editing

Instead of mapping a local folder into an nginx container, we could also edit the `index.html` file inside the container by means of a linux editor like `nano`.

- Start a plain nginx container: `docker run -p 8082:80 -d --name editable-webserver nginx`
- Execute a bash terminal in the container: `docker exec -it editable-webserver bash`
- Navigate to the folder containing the index.html: `cd /usr/share/nginx/html`
- Try to edit this file: `nano index.html`
 - `nano` is not installed in the very slim base image for nginx, hence we need to install it first ...
 - `cat` or `more` would be available, but this doesn't help
- Install nano:
 - First, package lists must be updated, since the base image doesn't contain any: `apt update`
 - Next, we can install nano: `apt install nano`.
 - Note that apt install asks for confirmation. Auto-answer with yes with the option `-y`: `apt install -y nano`
- Now edit index.html with nano, save the changes and reload the page in the browser.

2.1.3.2 Step 2: Save modified container to image

For each new container started from the base image `nginx`, those changes would be lost. But we can store the modifications done in the container to a new image layer:

- Exit the container bash
- Commit the container changes to a new image (`docker commit`): `docker commit -m "change-description" <container-id> <image-name>`
 - where image-name could be `nginx-editable`
- Inspect the new image with `docker images` and note the difference in size compared to the original base image `nginx`, resulting from the nano package installed.

2.1.3.3 Step 3: Start a new container

- Run a new container from the new image: `docker run -p 8083:80 -d nginx-editable`
- Test it on port 8083
 - You should see the modified index.html
 - If executing a bash inside the new container, you can use nano to edit files

2.1.4 Dockerfile

The most common way to build an image is using a Dockerfile. See the [Dockerfile reference](#).

2.1.4.1 Dockerfile Syntax

Instruction	Description	Example
FROM	A new image is always built based on another image, at minimum FROM scratch . This reference image will be extended with the given Dockerfile	FROM <code>nginx</code>
RUN	One or more shell commands to be executed while building the image. Referring to the previous example, we could do the package update and install commands here. Since each RUN statement results in a distinct image layer, we can save effort by combining several commands with <code>&&</code> . In Dockerfiles, <code>apt-get</code> is preferred over <code>apt</code> , since <code>apt</code> is meant for interactive use.	RUN <code>apt-get update && apt-get install -y nano</code>
WORKDIR	Sets the working directory for all subsequent instructions (and for the running container). If the folder does not yet exist, it will be created.	WORKDIR <code>/usr/share/nginx/html</code>

Instruction	Description	Example
COPY	To make specific files or folder appear in a future container of the new image, they must be copied from the build context (the folder passed to docker build) into the image. For example, a prepared web root folder can be copied into the nginx html folder. Source paths are always relative to the build context, destination paths are relative to the WORKDIR of the image (unless absolute). So COPY . . copies the whole build context into the current working directory of the image.	COPY . .
EXPOSE	A Dockerfile might declare the container port on which the service will be listening after startup. This does not publish the port, it mainly serves as documentation. This is very useful for a successful port mapping in the docker run command, since the right part of a port mapping must refer to the container port.	EXPOSE 80 (e.g., for a typical web server)
CMD	Usually the last instruction in a Dockerfile, which specifies the default command to start the application. This command is executed each time a container of this image is started, unless another command is passed to docker run . The preferred exec form is written as JSON array.	CMD ["node", "app.js"] (for a node application)

2.1.5 Dockerfile Exercises

1. [Hello World](#)
2. [Static Web Page](#)
 - Self-check: [quiz on Dockerfile basics](#) (overview, exercises 1 and 2)
3. [Javascript Express Backend](#)
4. [Java Server Multistage Build](#)
 - Self-check: [quiz on app images and multi-stage builds](#) (exercises 3 and 4)
5. [Typescript Backend](#)
6. [Typescript Frontend hosted with nginx](#)
 - Self-check: [quiz on the TypeScript backend and frontend images](#) (exercises 5 and 6)

2.2 First Dockerfile

2.2.1 Dockerfile

```
FROM alpine
CMD ["echo", "hello world"]
```

Note:

- CMD is written here in the recommended **exec form** (JSON array). It could also be written in **shell form**:
CMD echo hello world, which runs the command via `/bin/sh -c`.

2.2.2 Build Image

- Navigate to the folder where the Dockerfile resides
- Run:

```
docker build -t my-hello-world .
|      |      |
|      |      +--- Current directory is passed as build context:
|      |      contains all files/folders which shall be copied,
|      |      and (by default) the Dockerfile
|      +----- Name (and optional tag, e.g. my-hello-world:1.0)
|      for the new image; the tag defaults to `latest`
+----- docker command for building an image from a Dockerfile
```

- Inspect your image cache with `docker images`

2.2.3 Run a new Container

- Run a container from this image with `docker run my-hello-world`
- Notes:
 - no port mapping since the image is not a long-running server application
 - no detached mode (`-d`) since we want to see the output in the terminal
 - the container exits immediately after the echo command is completed (see CMD)

2.3 Publish Static Web Page

2.3.1 Dockerfile

```
FROM nginx
WORKDIR /usr/share/nginx/html
COPY public .
```

Note:

- `EXPOSE 80` is already declared in the base `nginx` image.
- `COPY` refers on the source side to the folder `public` in the build context, on the destination side to the current working directory of the image which has been set correctly in the line above with `WORKDIR`.
- `CMD` is already declared in the base `nginx` image, we don't need to repeat it here.

2.3.2 Build Image

```
docker build -t my-static-page .
```

2.3.3 Run Container

```
docker run -p 8084:80 -d --name my-static-page-container my-static-page
```

2.3.4 Test

- Open `http://localhost:8084` in your browser: you should see the `Home` page.
- Click the links `Galery` (with six city pictures) and `Contact` - all files of the `public` folder are inside the image now.
- Unlike the bind mount in chapter 01, changing a file in `public` does **not** change the page. Try it! You have to build the image again and start a new container.
- Clean up: `docker rm -f my-static-page-container`

2.3.5 Check yourself

You have finished the first two Dockerfile exercises. Take the [self-check quiz on Dockerfile basics](#): 5 short questions about the chapter overview, Hello World and this exercise, answered in your own words (German or English). The AI gives you feedback on each answer, and you can ask follow-up questions. You can repeat the quiz as often as you like.

2.4 Javascript Express Backend

What do you need to build a simple node.js express backend service in your local development environment?

- You need a local Node.js installation
- You need to create a new node project
- You need a js source file
- You need to install express as dependency
- You need to launch the JavaScript program to start the service

All those steps can be packaged into a docker image, so there is no local Node.js installation required anymore
=> the app can run wherever docker is installed.

2.4.1 Dockerfile

- `FROM node:24` - Node.js is required. Always specify a version tag, otherwise you get whatever `latest` currently is.
- `WORKDIR /app` - Creates a new folder where we will store the app including the `node_modules`
- `COPY . .` - Copies `app.js`, `package.json` from the build context into the app folder of this image
- `RUN npm install` - Creates `package-lock.json` and `node_modules`
- `EXPOSE 3000` - Declares that our express service will listen on container port 3000 after startup
- `CMD ["node", "app.js"]` - Finally declares how to startup the express service

2.4.2 Build the Image

... you should know already how ...

- Check the image size with `docker images`. Why is it so big? (Keep the number in mind for the next exercises.)

2.4.3 Start a container

... you should know already how ... - publish the container port 3000, e.g. `-p 3000:3000`.

2.4.4 Test

- Open `http://localhost:3000` in your browser. It should display `Hello World!`
- `docker logs <container>` should show `Example app listening on port 3000!`

2.4.5 Dockerignore

Note the file `.dockerignore` which excludes files matching the given rules from the build context, so `COPY` (and `ADD`) cannot copy them into the image.

Hence we can avoid that files like `Readme.md` and the `Dockerfile` itself will be copied to the image, since they are not needed for running the application. Even more important: a local `node_modules` folder is excluded too, so it cannot overwrite the dependencies installed by `npm install` inside the image.

Refer to <https://docs.docker.com/build/concepts/context/#dockerignore-files>

2.5 Multistage Dockerfiles

2.5.1 Java EchoServer

This Java program launches a server listening on incoming connections on port 9001. It will answer with some greeting and then echoes all text communication which is sent from the client terminal.

What is relevant for dockerizing this service:

- Internal port for the port mapping: 9001
- How to compile a Java source file: `javac` compiler which is part of JDK
- How to run the Java Bytecode file to launch the program: `java` which is part of the java runtime `jre`

2.5.2 Dockerfile

Since a full JDK environment is much larger than the java runtime itself, we save effort and file size by separating the image build process into two distinct stages ([multi-stage build](#)):

1. Build Stage for compiling the `.class` bytecode file from the `.java` source file
2. Run Stage for preparing the execution environment for launching the bytecode file with `jre`

2.5.2.1 Stage 1: Build Stage

```
FROM eclipse-temurin:21-jdk AS builder
WORKDIR /app
COPY EchoServer.java ./
RUN javac *.java
```

Note:

- This build stage is named `builder`, so the next stage can refer to it.
- The former `openjdk` images are deprecated on Docker Hub, `eclipse-temurin` is a recommended replacement.
- This stage won't be part of the final image after the docker build is complete => image file size is minimized.

2.5.2.2 Stage 2: Run Stage

```
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --from=builder /app/EchoServer.class .
EXPOSE 9001
CMD ["java", "EchoServer"]
```

Notes:

- Final image only requires `jre` environment
- Bytecode file is copied from the previous build stage
- `EXPOSE 9001` documents the port the echo server listens on
- Startup command calls the `java` program and passes the class name `EchoServer` (without the `.class` extension) as an argument.

2.5.3 Build the Image

... you know how ...

2.5.4 Run a Container

... you know how ... - don't forget to publish port 9001, e.g. `-p 9001:9001`.

2.5.5 Test the program

- Launch a terminal and create a client connection with the telnet command: `telnet localhost 9001`
 - On Windows, telnet is not installed by default. Either enable the Windows feature “Telnet Client” or use another TCP client like PuTTY (connection type “Raw”).

2.5.6 Check yourself

You have finished exercises 3 and 4. Take the [self-check quiz on app images and multi-stage builds](#): 5 short questions about the Express backend and this Java multi-stage build, answered in your own words (German or English). The AI gives you feedback on each answer, and you can ask follow-up questions. You can repeat the quiz as often as you like.

2.6 TypeScript Backend Dockerfile

2.6.1 Fruits Rest-API

Given is the already known sample for the Fruits Rest-API with following changes:

- `requests.http`:
 - Introduced variable for `HOSTNAME` so we can edit the host name and port in a single place, if someone maps the container port to a different external port than 3000.
- `package.json`:
 - added build script for the build stage
 - installed missing cors dependencies
- `tsconfig.json`:
 - set `outDir` to `dist` folder so the js files will be transpiled into this folder.
 - Thus, we can keep the files required for launching the program in a separate folder, which makes copying easier.

2.6.2 Dockerfile

A typescript app involves a compilation (transpilation) step, which needs the full `node` image including `npm` and the TypeScript compiler. For launching the transpiled `app.js` file, however, only the Node.js runtime is required. Google's `distroless` images contain nothing but the runtime (no shell, no package manager), so the final image is only a fraction of the size of the full `node` image (check the difference with `docker images`).

- Refer to: <https://github.com/GoogleContainerTools/distroless>

So it definitely makes sense to split the Dockerfile into two stages.

2.6.2.1 Step 1: Build Stage

- `FROM node:24 AS builder`:
 - Full node image (including npm) for building
 - Write `AS` in upper case like `FROM`, otherwise docker build prints a `FromAsCasing` warning
- `WORKDIR /app`:
 - We will install and transpile into this folder
- `COPY ./package*.json ./`:
 - First, only copy `package.json` and `package-lock.json` (if present) from the build context into the image.
 - This reduces build time for later docker builds: docker caches each build step and reuses it, as long as the files copied in this step have not changed. So `npm install` only runs again if the dependencies changed.
- `RUN npm install`:
 - creates `node_modules` in app folder
- `COPY . .`:
 - Now, all the rest is copied from the build context into the image, including `tsconfig.json` and the ts source files.
 - The `.dockerignore` file makes sure that a local `node_modules` or `dist` folder is not copied, since it would overwrite the folders created inside the image.
- `RUN npm run build`:
 - Javascript files are transpiled into `/app/dist` folder

2.6.2.2 Step 2: Run Stage

- `FROM gcr.io/distroless/nodejs24-debian12`

- For running the javascript files, a small distroless image containing only the Node.js 24 runtime is used.
- `WORKDIR /app`
 - Again, we define a common base folder for the program
- `COPY --from=builder /app/dist .`
 - Copy js files
- `COPY --from=builder /app/node_modules ./node_modules`
 - Copy js dependencies
- `EXPOSE 3000`
 - Declare the port on which our express application listens
- `CMD ["app.js"]`
 - The base image for this stage already defines `node` as entrypoint for the startup command, therefore we only need to specify the path to the main js file in `CMD`
 - Note the `dist` folder only exists in the build stage, we have copied all contents of this folder from the build stage to the current working directory `/app`.

2.6.3 Build, run, test

```
cd src
docker build -t fruit-backend .
docker run -p 3000:3000 -d fruit-backend
```

Test the endpoints in VS Code with `requests.http`. This needs the VS Code extension [REST Client](#): click `Send Request` above a request.

- GET `http://localhost:3000/api/fruits` should return `["apple","banana","peach"]`.
- Quick check without VS Code: open `http://localhost:3000/api/fruits` in your browser.

2.7 Hosting Dynamic Website with nginx

2.7.1 Fruit Rest Client

Again we use the already known small rest client for consuming the Fruit Rest-API from a web page.

Following adaptations in the source code have been done:

- `public` folder added
 - Moved `index.html` into this folder
 - Changed `tsconfig.json` to generate the `index.js` transpilation output into this folder
- `package.json` file added
 - for installing typescript compiler in the docker node image
- `nginx.conf` file added
 - It overrides the default nginx configuration: every URL which does not match an existing file is answered with `index.html` (`try_files`). This is needed for single-page applications with client-side routing.

2.7.2 Dockerfile

The setup is again separated into two stages:

1. We need to use `node` as base image for generating the transpilation output `index.js`.
2. For the run stage, we don't use `node`, but `nginx` as web server, which hosts both the html and javascript files => therefore we put all our application files into the common `public` folder. This folder will then be hosted by nginx.

2.7.2.1 Step 1: Build Stage

The build stage is basically the same as for the typescript backend application:

```
FROM node:24 AS builder
WORKDIR /app
COPY ./package*.json ./
RUN npm install
COPY . .
RUN npm run build
```

2.7.2.2 Step 2: Run Stage

Since we have created all what nginx needs for hosting the web page into the folder `/app/public` in the build stage, the only thing we need to do is to copy this folder into the web root folder of nginx: `/usr/share/nginx/html`.

```
FROM nginx
COPY ./nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=builder /app/public /usr/share/nginx/html
```

Note:

- No `EXPOSE 80` or `CMD` required, these statements are already declared in the nginx base image.
- Overriding the `default.conf` is only required if your web application uses client-side routing (a single-page application with several URL routes). A plain static site works with the default configuration.

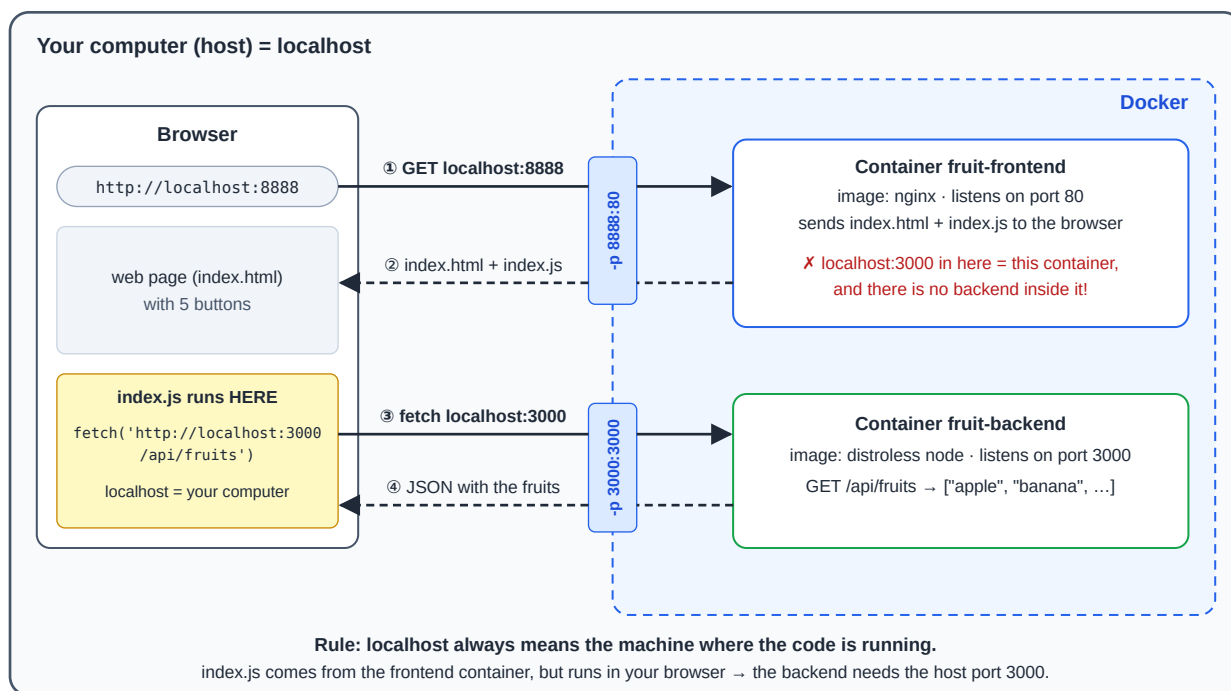
2.7.3 Build, run, test

```
cd src
docker build -t fruit-frontend .
docker run -p 8888:80 -d fruit-frontend
```

Test the URL `http://localhost:8888` in your browser.

- You should see five buttons. With the backend from the previous exercise running on port 3000, `GET /api/fruits` shows **apple,banana,peach**.
- Now stop the backend container and click the buttons again. Most buttons show nothing new on the page (the two `.../0` buttons show **fruit not found**). Open the browser developer tools (F12 → tab **Console**) to see the real error messages.
- The JavaScript code runs in your browser, not in the container. Hence it calls the backend via `http://localhost:3000`, and the backend container must publish port 3000 on your host.

2.7.3.1 Browser localhost vs. container localhost



1. The browser asks your computer for `localhost:8888`. Docker forwards this request to port 80 of the frontend container (nginx).
2. nginx sends back `index.html` and `index.js`. From now on, **nobody** in the frontend container runs this code - it runs in your browser.
3. When you click a button, `index.js` calls `http://localhost:3000/api/fruits`. For the browser, `localhost` is your computer, so the request goes to host port 3000, which Docker forwards to the backend container.
4. The backend answers with JSON.

Remember: `localhost` always means the machine (or container) where the code is running. Inside a container, `localhost` is the container itself. The frontend container has no backend on port 3000, so it would not help to call the backend from there.

Question: What happens if you start the backend with `-p 3001:3000`? What would you have to change?

2.7.4 Check yourself

You have finished exercises 5 and 6 (think about the question above first). Take the [self-check quiz on the TypeScript backend and frontend images](#): 5 short questions, answered in your own words (German or English). The AI gives you feedback on each answer, and you can ask follow-up questions. You can repeat the quiz as often as you like.

Part 3

Docker Compose

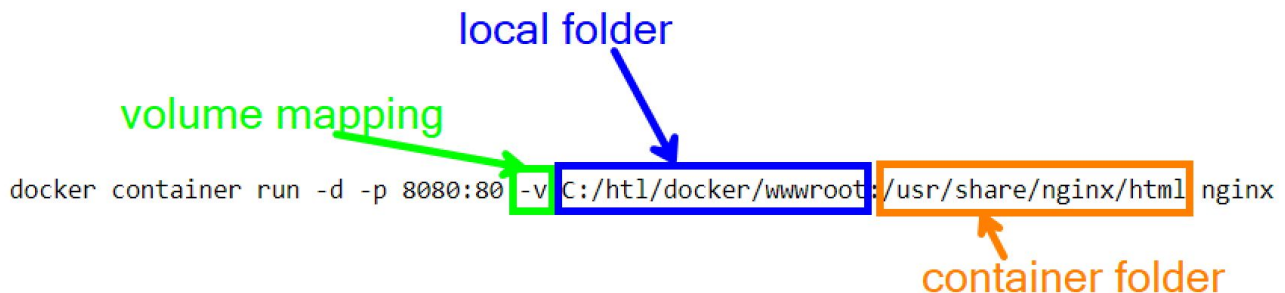
3.1 Container Orchestration with docker compose

Stuck or have a question? Ask the [AI tutor for this chapter](#). It knows this chapter and all its exercises, and helps you with compose files, networks, volumes and error messages. For the exercises where you write the compose file yourself, it gives you hints, not the solution. You can ask in German or English.

1. [Compose a single container](#)
2. [Compose multiple containers and networks](#)
 - Self-check: [quiz on compose basics and networks](#) (exercises 1 and 2)
3. [DB containers and persistent volumes](#)
4. [Wordpress, phpMyAdmin, mariadb](#)
 - Self-check: [quiz on volumes, databases and WordPress](#) (exercises 3 and 4)
5. Full Typescript Stack (coming soon)

3.2 Composing a single container

In the container section, we have tested a docker run command for publishing a local folder through an nginx container and mapping the internal nginx port 80 to the external host port 8080:



The diagram shows the command `docker container run -d -p 8080:80 -v C:/htl/docker/wwwroot:/usr/share/nginx/html nginx`. Annotations include: a green arrow labeled "volume mapping" pointing to the `-v` flag; a blue arrow labeled "local folder" pointing to `C:/htl/docker/wwwroot`; an orange arrow labeled "container folder" pointing to `/usr/share/nginx/html`; and a blue box around the entire volume mapping `-v C:/htl/docker/wwwroot:/usr/share/nginx/html`.

With `docker compose`, we can specify such docker run configurations in a text file and use this file to start and stop (or remove) a single or multiple containers.

Refer to:

- <https://docs.docker.com/reference/compose-file/>

3.2.1 compose.yaml

The default file name for a compose file is `compose.yaml` (preferred) or `compose.yml`. The older names `docker-compose.yaml` and `docker-compose.yml` are still supported for backwards compatibility.

Note:

- Older compose files often start with a `version:` element. It is obsolete and should be omitted.

`services:`

- First statement is - with no indentation, starting at the very first character - the `services:` section, which allows to specify one or more containers (=> `services`) to be managed.
- After this line, you need to indent with spaces (usually 2)! YAML does not allow TAB characters for indentation.

`web:`

- Next, we declare the service name `web` for the one and only container, which will be managed by this compose file.
- All subsequent mappings for this service are again indented (2 more spaces)

`image: nginx`

- The most important service mapping is of course, on which image this container should be based.
- So we define `nginx` with no tag, thus, the tag `latest` is used (which usually points to the newest stable version).

Note:

- Instead of an image, which would be pulled from Docker Hub, we could also specify image build options with a `build` mapping, since docker compose can also involve a docker build to create the container image on the fly.
- More on this later.

`ports:`

`- 8080:80`

- A container usually represents a background server process and listens on one or more ports. These are called the container ports.
- Hence, port mappings are specified in form of a list with each element started with `-` in a distinct row.
- In our example, the `nginx` container port `80` is published on the host port `8080`, similar to the port mapping we have done with the `docker run` command in the terminal.

```
volumes:
  - ./wwwroot:/usr/share/nginx/html
```

- Finally, the bind mount of a local host folder to the default `nginx` web root folder is configured as one element for the `volumes` mapping, very similar to the `-v` option of the `docker run` command.
- Note that `compose` resolves a relative path relative to the folder of the `compose` file.

3.2.2 Startup: `docker compose up`

If you `cd` into the directory, where the `compose.yaml` file resides, and call the `docker compose up` command there, you will first get following information in your terminal:

```
- Network 01-compose-single-container_default Created
- Container 01-compose-single-container-web-1 Started
```

Following two steps happened:

1. `docker compose` creates a separate `docker` network for you.
 - As long as no specific `docker` network is configured in the `compose` file, this network is called `<project-name>_default`. The project name defaults to the name of the folder containing the `compose` file.
 - You can inspect this network with the command `docker network ls`, listing both the default `docker` networks and specific networks created manually or by means of `docker compose`:
 - `bridge` (where containers started manually with `docker run` will reside)
 - `host` and `none` for special network use cases
 - and now also the `01-compose-single-container_default` network.
 - Note the `NETWORK_ID` of your new network.
2. `docker compose` started the single service as specified in the `compose` file for you:
 - We did not ask for a specific container name, hence the project name (= folder name) is used, followed by the service name `web` and the instance number `1`.
 - We did not assign this container to a specific network, so it is assigned to the default network belonging to this `compose` file.

3.2.2.1 Network Inspection

You can inspect your `docker` network with `docker network inspect <network-id>`:

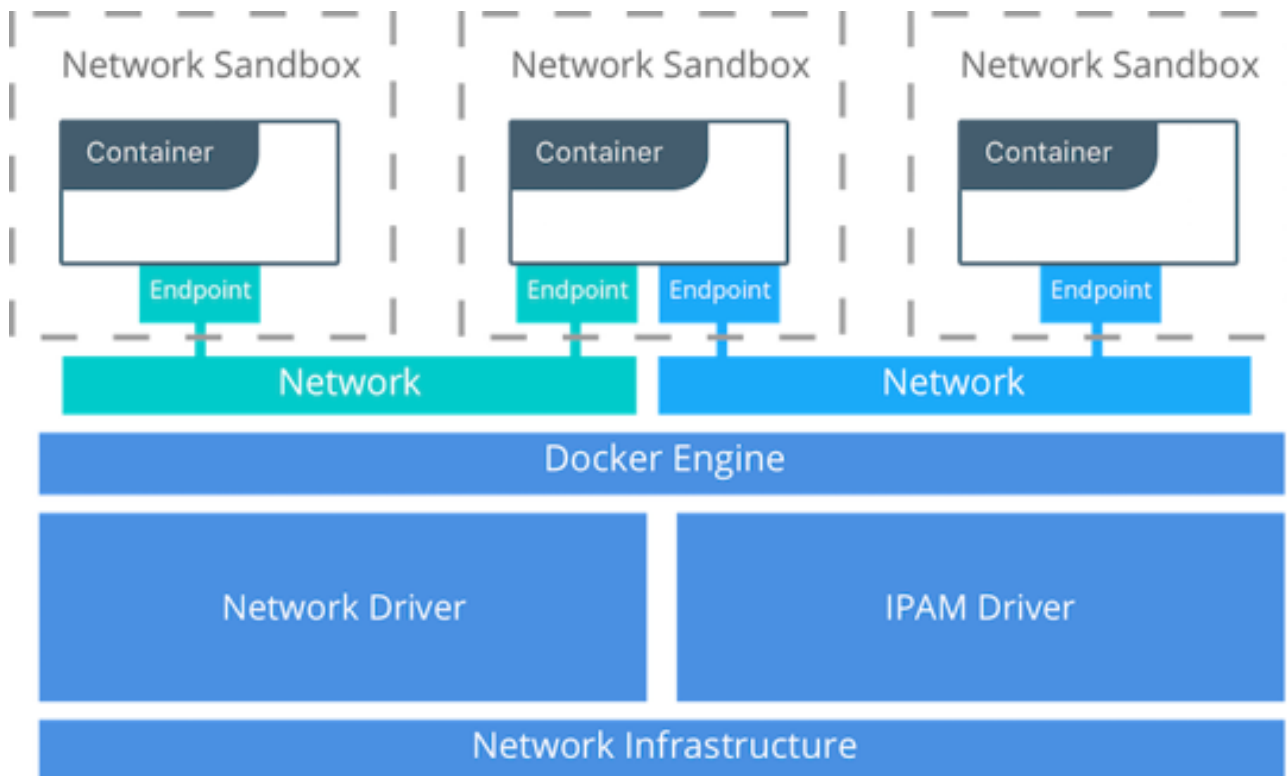
```

PS C:\htl\2223\2223-wmc-3\Units\XX-docker> docker network inspect d1d
[
  {
    "Name": "01-compose-single-container_default",
    "Id": "d1dfcd4c9a0df74cfb3fbb9949e3f2cde4f6eabf427d5184f246ebaac1b395f2",
    "Created": "2022-08-10T10:29:26.944028117Z",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": null,
      "Config": [
        {
          "Subnet": "172.26.0.0/16",
          "Gateway": "172.26.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Containers": {
      "d5a72d0b4a72a80db3416b10db506d762d27eb5348802d303bf9054d39381a51": {
        "Name": "01-compose-single-container-web-1",
        "EndpointID": "00b0306beb7b142fa159269a24e910c4b86ec4d3c0c08e7bd5e20593d15c8b8e",
        "MacAddress": "02:42:ac:1a:00:02",
        "IPv4Address": "172.26.0.2/16",
        "IPv6Address": ""
      }
    },
    "Options": {},
    "Labels": {
      "com.docker.compose.network": "default",
      "com.docker.compose.project": "01-compose-single-container",
      "com.docker.compose.version": "2.2.3"
    }
  }
]

```

Here you can see following:

- Network properties like the **Name**
- IP configuration for this network including
 - subnet
 - gateway
- All containers which have been assigned to this network including their IP address.
 - As you can see in the graphic below, a container can also be assigned to more than one docker networks:



See also:

- <https://docs.docker.com/engine/network/drivers/bridge/>

3.2.2.2 Background versus Foreground

Since we started our docker compose in foreground mode, the terminal is attached to the log outputs of all containers of this compose project.

To stop the containers,

- you can either press **Ctrl+C** in the terminal, which will stop but not remove the containers.
- or you open another terminal, cd into the compose folder and type **docker compose down**. In this case, the containers are not only stopped, but also removed, together with the networks created by compose.

3.2.2.3 Background mode: `docker compose up -d`

To start the containers in background (detached) mode, use the **-d** option of the **up** command: **docker compose up -d**. Note that **-d** must be written **after up**, since it is an option of the **up** sub-command.

Refer to <https://docs.docker.com/reference/cli/docker/compose/up/>

3.3 Exercise: Compose container network

Two different web root folders are given

- webA with an index.html telling that this is the A-service
- webB for the B-service

3.3.1 Instructions

You shall create a file `compose.yaml` in **this** folder (next to `webA` and `webB`) for managing two `nginx` containers:

- The services should be called `web-1` and `web-2`.
- The first should have the `container_name` `webA`, the second `webB`.
- The first should be mapped to port `8081`, the second to `8082`.
- The first should serve the folder `webA` (bind mount, like in the previous exercise), the second the folder `webB`.
- Both should be assigned to a specific network called `my-web-network`.
- This network should specify a specific IPAM config with the subnet `10.2.0.0/16`

Hints:

- Networks are declared in a top-level `networks:` section, similar to `services:`. Look up the syntax in the compose reference: [networks](#) (see `name`, `driver` and `ipam`).
- Each service lists the networks it belongs to in its own `networks:` list.
- Without `name:`, compose would put the folder name in front of the network name.
- Test: `http://localhost:8081` should show the A-service, `http://localhost:8082` the B-service.

Only if you are stuck: a solution is in [solution/compose.yaml](#). Start it from this folder with `docker compose -f solution/compose.yaml up -d` (and stop it with `docker compose -f solution/compose.yaml down`).

3.3.2 Expected Terminal Output

3.3.2.1 `docker compose up -d`

```
- Network my-web-network Created
- Container webB Started
- Container webA Started
```

3.3.2.2 `docker ps`

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
60d5c2755d53	nginx	"/docker-entrypoint...."	6 minutes ago	Up 6 minutes	0.0.0.0:8082->80/tcp, [::]:8082->80/tcp
c1e104106b4b	nginx	"/docker-entrypoint...."	6 minutes ago	Up 6 minutes	0.0.0.0:8081->80/tcp, [::]:8081->80/tcp

3.3.2.3 `docker compose down`

```
- Container webA Removed
- Container webB Removed
- Network my-web-network Removed
```

3.3.3 Inter-Container-Communication

Both containers reside in the same user-defined bridge network, and docker provides automatic DNS resolution for such networks. This means containers inside the same network can contact each other using either their IP addresses or their host names (=> which are actually their service and/or container names)!

Note:

- This DNS resolution only works in user-defined networks (like the ones created by compose). Containers in the default `bridge` network can only reach each other by IP address.

- Refer to <https://docs.docker.com/engine/network/drivers/bridge/>

We can experience this by using the `ping` utility.

3.3.3.1 Execute bash in webA

We connect our terminal to the webA container with

```
docker exec -it webA bash
```

However, the `ping` command is not found. So we need to install it in our container:

```
apt update
apt install -y iputils-ping
```

Finally, we are able to `ping orf.at` and get according ping statistics.

3.3.3.2 ping webB

Now we can ping our neighbour container named `webB` and get the following output:

```
root@a4074b8a929b:/# ping webB
PING webB (10.2.0.2) 56(84) bytes of data.
64 bytes from webB.my-web-network (10.2.0.2): icmp_seq=1 ttl=64 time=0.066 ms
```

(Your IP address may be different, e.g. 10.2.0.3 - docker picks a free address from the subnet. Stop `ping` with `Ctrl+C`.)

Also working:

- `ping 10.2.0.2` (using the internal ip address shown in your ping output)
- `ping web-2` (using the service name instead of `container__name`)

Question: After `docker compose down` and `docker compose up -d`, is `ping` still installed in `webA`? Why (not)?

3.3.4 Check yourself

You have finished compose exercises 1 and 2 (think about the question above first). Take the [self-check quiz on compose basics and networks](#): 5 short questions, answered in your own words (German or English). The AI gives you feedback on each answer, and you can ask follow-up questions. You can repeat the quiz as often as you like.

3.4 DB Containers / Persistent Volumes

Especially for hosting a database container, the data managed by the DBMS should survive a container life cycle. This means, if someone shuts down the container with `docker compose down` or manually removes the database container with `docker rm -f ...`, we would like to see the same data after launching a new database container. For this, we need to map the file path inside the db container, where all the database files are stored internally, to a **volume**, which persists independently of the container.

This file path depends on the DBMS, for instance for mysql it is `/var/lib/mysql`

3.4.1 Volume declaration

Similar to a service or network declaration, a volume declaration in a compose file looks like this:

```
volumes:
  data:
    name: mysql-data
```

In a terminal, the corresponding command would be `docker volume create mysql-data`. Compose creates the volume automatically, if it does not exist yet.

Docker-managed volumes can also be inspected and deleted manually:

- `docker volume ls`
- `docker volume inspect mysql-data`
- `docker volume rm mysql-data` (be careful!)

3.4.2 Volume mapping for a mysql container

```
services:
  mysql:
    image: mysql:8.4
    container_name: dev_mysql
    environment:
      MYSQL_USER: user
      MYSQL_PASSWORD: user
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: userDB
    volumes:
      - data:/var/lib/mysql
```

With this volume mapping, the container internal path `/var/lib/mysql` is mapped to the docker-managed volume `data` (named `mysql-data`, according to the volume declaration above).

Note:

- Still, you could map a local host path on the left side, by specifying a relative or absolute file path instead of the name of a volume.
- This is called a **bind mount**, whereas mapping to a docker-managed volume is called a **named volume**.
- In the first case, the files are accessible very easily, you can browse them in the local file system. But this is also more dangerous, since you could delete some files by accident.
- In the second case, the files are managed by docker and stored in a part of the host file system which is managed by docker (`/var/lib/docker/volumes/` on Linux, inside the Docker Desktop VM on Windows/macOS), so it is not easy at all to delete them by accident.

Refer to:

- <https://docs.docker.com/engine/storage/> (overview of volumes, bind mounts and tmpfs mounts)
- <https://docs.docker.com/engine/storage/volumes/>

3.4.3 phpMyAdmin

With the DNS resolution inside a docker network, a good practice for database management in docker is to **hide** a DBMS docker container from the external world, while only neighbour containers inside the same docker network are able to contact this DBMS.

Note:

- During development, you might still want to publish the database port, so you can inspect your tables with a database tool on your host (see exercise below).

We test this security feature by setting up a second container using the `phpmyadmin` image, which is basically a web server providing a user interface for browsing a relational database. This web service uses PHP to communicate with the DBMS container, and for this, network traffic inside the docker network is sufficient!

Note:

- That's the reason why the `mysql` container does not specify a port mapping, hence it is not reachable from outside!

3.4.3.1 pma Container

```
phpmyadmin:
  image: phpmyadmin
  container_name: dev_pma
  environment:
    PMA_HOST: mysql
    PMA_PORT: 3306
    PMA_ARBITRARY: 1
  restart: always
  ports:
    - 8183:80
```

During startup of a `phpMyAdmin` container, this application refers to several environment variables. Those can easily be specified in a compose file:

- `PMA_HOST`: Hostname or IP address of the DBMS which shall be browsed
- `PMA_PORT`: By default, a `mysql` database listens on port 3306 for incoming client connections
- `PMA_ARBITRARY`:
 - If set to 0, the web interface is fixed to the given `PMA_HOST`. In the login dialog, you are only asked for username and password.
 - If set to 1, the web interface is allowed to connect to arbitrary DBMS hosts. In the login dialog, you can also enter the `PMA_HOST` which shall be connected, in addition to username and password.

3.4.4 First Booting

If the compose file is started for the first time, `mysql` detects an empty data directory and creates the requested database and user (refer to environment variables passed to the `mysql` container).

Note:

- These environment variables are only evaluated if the data directory is empty. Changing them later has no effect on an existing volume.

Since the docker-managed volume survives several downs and ups of the compose network, all data remain there, until someone manually deletes the volume in a terminal with `docker volume rm mysql-data`, or with `docker compose down -v` (which removes the containers together with the volumes declared in the compose file).

3.4.5 Exercise

3.4.5.1 Database hidden

- Boot the compose network

- On the very first start, MySQL needs some time (up to about a minute) to create the database. Until then, the login fails. Watch `docker logs -f dev_mysql` until you see `ready for connections` together with `port: 3306` (stop watching with `Ctrl+C`). An earlier line with `port: 0` belongs to a temporary setup server - keep waiting.
- Test the web interface in your favourite browser : `http://localhost:8183`
 - Login with `root/root` or `user/user`
 - Test various host names in the **Server** field of the login dialog: `mysql` (service name), `dev_mysql` (container name), the IP address of the container (find it with `docker network inspect backend`) and `localhost`.
 - Why does `localhost` **not** work? (Hint: inside the phpMyAdmin container, `localhost` means the phpMyAdmin container itself.)
- Create a new table with some columns
- Shut down the network with `docker compose down`.
- Boot again, login to the database in the browser and inspect the new table.

3.4.5.2 Database with external port mapping

- Try to connect to the database from an external sql browser (like JetBrains DataGrip, HeidiSQL or similar) => should not be possible.
- Change the compose file and add a port mapping for the mysql container (remove the `#` in the commented `ports` lines), then run `docker compose up -d` again.
 - If you get `port is already allocated`, another MySQL (e.g. from XAMPP) is running on your computer. Stop it, or use another host port, e.g. `3307:3306`.
- Try again to connect to the database from an external sql browser => should work now! (Host `localhost`, port `3306`, user `user`, password `user`)
- Clean up: `docker compose down` (the volume `mysql-data` stays; `docker compose down -v` deletes it too)

Note: Passwords written directly into a compose file are fine for a local development setup, but never for a real server.

3.5 Wordpress with docker compose

This example manages three different containers in a compose file:

- A mariadb DBMS container, its data path is mapped to a named volume
 - Note: the **service** is called `mysql`, but the **image** is `mariadb`. MariaDB is a fork of MySQL and works as a drop-in replacement, so it also uses the `MYSQL_...` environment variables and port 3306.
- A phpMyAdmin container, which can be used to browse the mariadb
- A wordpress container, which is basically an apache2 web server with a lot of PHP and web files installed. With php, this web server connects to the mariadb and stores all the content which wordpress users add to the wordpress blog.

No **networks:** section is given here, so all three containers are placed in the default network of this compose project (`04-wordpress-network_default`). They can still reach each other by service name.

3.5.1 First boot

- Before you start: run `docker compose down` in the folders of the previous compose exercises. This example uses the same host ports (8080, 8183) and the same container name `dev_pma`.
- Start with `docker compose up -d`.
- Open `http://localhost:8080`. If the wordpress container is visited the first time, it will ask for an initial installation, including the database connection data.
- phpMyAdmin is available at `http://localhost:8183` (user `wordpress`, password `wordpress`).

3.5.2 Exercise

Find out, how you can configure the wordpress container, so the initial installation does not ask for database connection data anymore.

Hints:

- Look at the documentation of the image on Docker Hub: https://hub.docker.com/_/wordpress, section **How to use this image**. Look for environment variables starting with `WORDPRESS_DB_`.
- Which values do you need? Compare with the **environment** section of the `mysql` service: host name, user, password, database name.
- After changing the compose file, run `docker compose up -d` again. Compose recreates only the changed container.
- If the page shows an error (**Error establishing a database connection** or HTTP 500) right after the start, the database is still starting. Wait a few seconds and reload.

Expected result: `http://localhost:8080` skips the database step. After choosing the language, it directly asks for the site title, admin user name and password.

3.5.3 Check yourself

You have finished compose exercises 3 and 4. Take the [self-check quiz on volumes, databases and WordPress](#): 5 short questions, answered in your own words (German or English). The AI gives you feedback on each answer, and you can ask follow-up questions. You can repeat the quiz as often as you like.